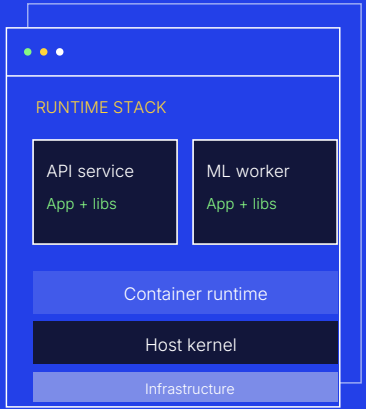
WEEK 2 / LECTURE 03

MLOps: Machine Learning Operations

Containerisation, Virtual Machines & Cloud Infrastructure

Isolate your software.
Run it consistently across machines.

TRAN Cong Minh



Three boundaries of isolation

Similar goal, different isolation boundaries: packages, operating systems, and processes.

01

Virtual environments

DEPENDENCIES

Project-specific Python libraries and interpreter settings.

02

Virtual machines

FULL GUEST OS

Virtual hardware with a complete operating system.

03

Containers

APPLICATION PROCESS

Packaged runtime sharing a compatible host kernel.

Ask: which boundary would prevent a dependency conflict? Which one can isolate an entire OS?

PART 1 / CLOUD INFRASTRUCTURE

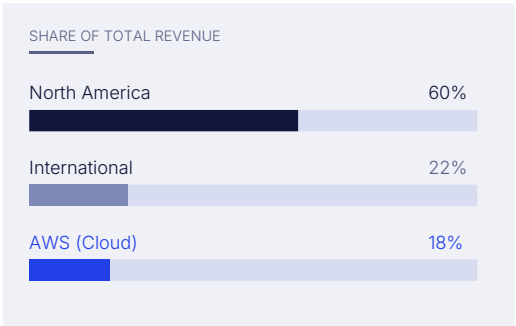
Cloud infrastructure as a service.

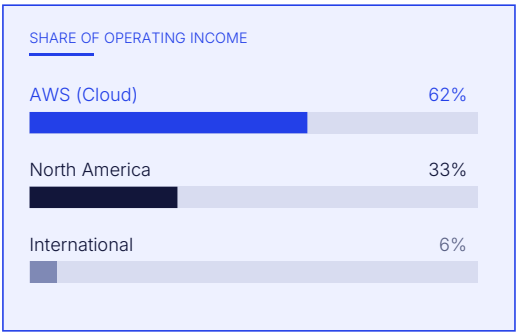


Computing, storage, and managed services available on demand.

Cloud is a profitable business

Amazon Q1 2024 segment mix (quarter ended March 31). Source: Amazon Investor Relations, Apr 30, 2024.



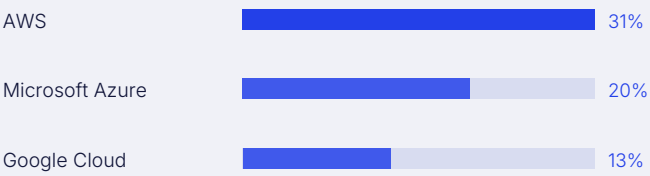


AWS was 18% of reported net-sales mix and about 62% of operating income in Q1 2024.

Major cloud infrastructure providers

Worldwide cloud infrastructure services, Q3 2024 (historical snapshot).

GLOBAL REVENUE SHARE



Other providers account for approx. 36% combined.

Q3 2024 SPENDING

~$84B

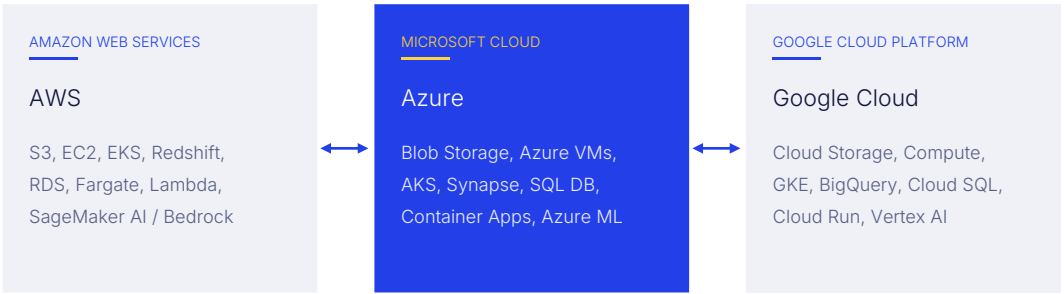
+23% y/y

Top 3: approx. 64% of global cloud infrastructure services.

Cloud infrastructure market share is not a measure of MLOps deployment share.

Cloud providers offer really similar services

Different product names, same underlying architectural building blocks.



Learn the service category and container concepts once; map them to any provider.

The three major clouds

Same building blocks under three brands; examples from the services used in this course.

AMAZON WEB SERVICES



S3 · EC2 · EKS
SageMaker AI · Bedrock

MICROSOFT CLOUD



Blob Storage · Virtual Machines
AKS · Azure Machine Learning

GOOGLE CLOUD



Cloud Storage · Compute Engine
GKE · Vertex AI

Learn the service categories once; map them to any provider.

Logos: Wikimedia Commons (Amazon Web Services Logo.svg, Apache-2.0; Microsoft Azure Logo.svg and Google Cloud logo.svg, public domain). Trademarks belong to their owners.

Cloud service models and functions

01	Infrastructure as a Service (IaaS)	On-demand pay-as-you-go storage, VMs and networking.
02	Platform as a Service (PaaS)	Managed hardware/software for apps, databases and ML.
03	Software as a Service (SaaS)	Cloud-hosted end-user software and BI dashboarding tools.
04	Function as a Service (FaaS)	Serverless functions triggered by events or HTTP.

NIST defines IaaS, PaaS and SaaS; FaaS is an additional serverless execution pattern.

Infrastructure as a Service (IaaS)

On-demand "pay-as-you-go" data, compute and networking infrastructure.

DATA STORAGE / RAW BLOB STORAGE

AWS:	AWS S3
Google:	Cloud Storage (GCS)
Azure:	Azure Blob Storage

- Store flexible files (docs, images, videos, logs).
- Accessible via Python SDK; pass unstructured data between application components.

COMPUTE / VIRTUAL MACHINES

AWS:	AWS EC2
Google:	Compute Engine
Azure:	Azure VMs

- Full control over OS, software & configuration.
- Provider manages physical hardware; you can SSH into the VM and run what you want.

IaaS gives maximum flexibility over the operating system while abstracting physical data centers.

Managed compute: apps and Kubernetes

Delivers and manages hardware and software resources for developing, testing and deploying cloud apps.

APPLICATION DEPLOYMENT

AWS:	Elastic Beanstalk
Google:	App Engine
Azure:	App Service

- Build and host web apps, APIs & mobile backends.
- Often used for front-end applications; underlying infrastructure is managed by the cloud provider.

KUBERNETES HOSTING (K8S)

AWS:	Elastic Kubernetes Service (EKS)
Google:	Google Kubernetes Engine (GKE)
Azure:	Azure Kubernetes Service (AKS)

- Managed Kubernetes control plane; node options vary.
- Deploy microservices; Kubernetes supports pod autoscaling when configured.

Managed Kubernetes reduces control-plane work; teams still configure workloads, nodes and scaling.

Managed data: analytics, SQL and NoSQL

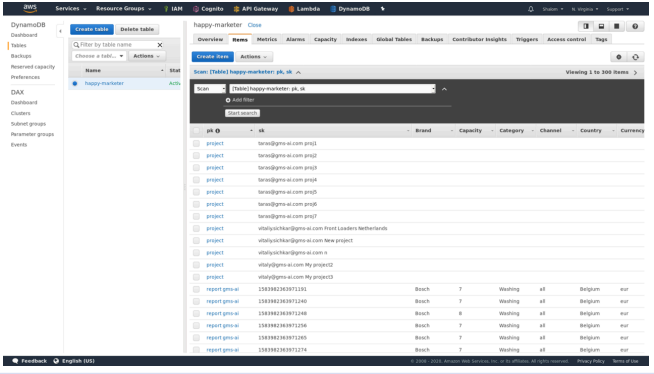
Managed database services tailored to analytics, transactions or low-latency semi-structured data.

DATABASE TYPE	AWS / GOOGLE / AZURE	ROLE & ML USE CASE
SQL analytics / warehouse (OLAP)	`Redshift / BigQuery / Synapse Analytics`	Store data and run large-scale aggregations; heavily used in ML pipelines for analytics & features.
Transactional relational (OLTP)	`RDS / Cloud SQL / Azure SQL Database`	Managed relational SQL services for operational transactions and structured queries.
NoSQL databases	`DynamoDB / Firestore / Cosmos DB`	Key-value / document services; models vary Scaling and global access depend on setup.

Match the database engine to the workload: analytical queries, ACID transactions or key-value lookups.

A cloud console in practice

Browsing a DynamoDB table in the AWS Management Console.



A console is a web UI over the same APIs you can call from code or the command line.

Screenshot: Wikimedia Commons, “Dynamodb-aws-console.png”, Vnon, CC0.

Platform as a Service: Machine Learning

Managed services support the ML lifecycle, from experiments to deployed endpoints.

AWS

SageMaker AI

Build, train, evaluate, register and deploy ML models. Bedrock is a separate service focused on foundation models.

GOOGLE

Vertex AI

Managed development, training, pipelines, endpoints, experiments and monitoring.

AZURE

Azure Machine Learning

Workspaces, compute, pipelines, model registry, endpoints and MLOps workflows.

Service catalogues differ; the lifecycle concepts transfer between providers.

Software as a Service (SaaS)

Applications you use through a hosted service rather than managing their infrastructure.

ANALYTICS

Tableau Cloud

Hosted/managed BI dashboards and visual analytics; a Salesforce product.

GOOGLE CLOUD

Looker

Semantic modelling, dashboards, exploration and business intelligence.

MICROSOFT

Power BI

Reports, dashboards and sharing in the Power BI service.

SaaS describes the delivery model; these are not always products of the same IaaS provider.

Serverless compute: functions and containers

Serverless is an operational model; FaaS is one way to execute on demand.

FUNCTION AS A SERVICE (FAAS)

Event-driven functions

- AWS Lambda · Google Cloud Run functions · Azure Functions
- Invoke code on demand in response to events, HTTP requests or schedules.

SERVERLESS CONTAINERS

Managed container runtime

- Amazon ECS on Fargate · Google Cloud Run · Azure Container Apps
- Run containerized services and jobs without managing underlying servers.

Both abstract server provisioning. They differ in packaging, execution model and scaling controls.

Out-of-the-box AI services

Use pre-trained models through cloud APIs; not every AI workload requires training.

GENERATIVE AI

Foundation models

Managed access to LLMs, multimodal models and embedding APIs.

DOCUMENTS

Vision & OCR

Extract printed text, parse documents and classify images.

LANGUAGE

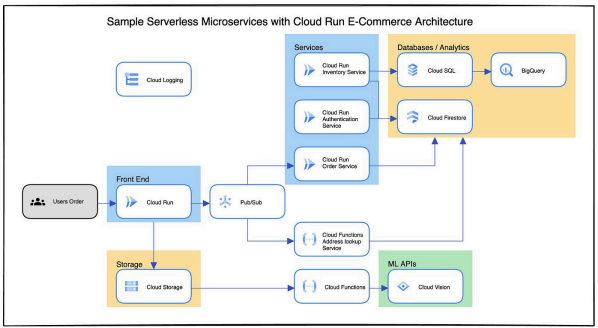
Speech & translation

Speech-to-text, translation and language analysis as APIs.

Compare hosted model APIs with deploying and monitoring your own trained model.

Reference architecture: Google Cloud

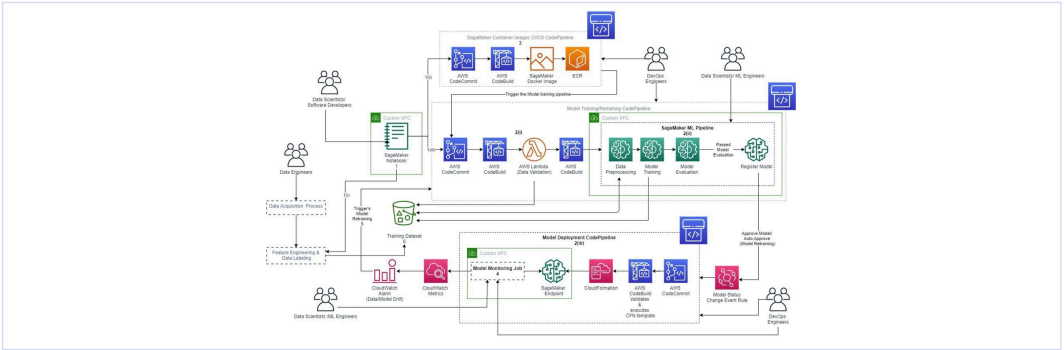
Serverless e-commerce services composed from Cloud Run, Pub/Sub, storage and data products.



Follow an order from the front end through events, services and databases.

Reference architecture: AWS MLOps

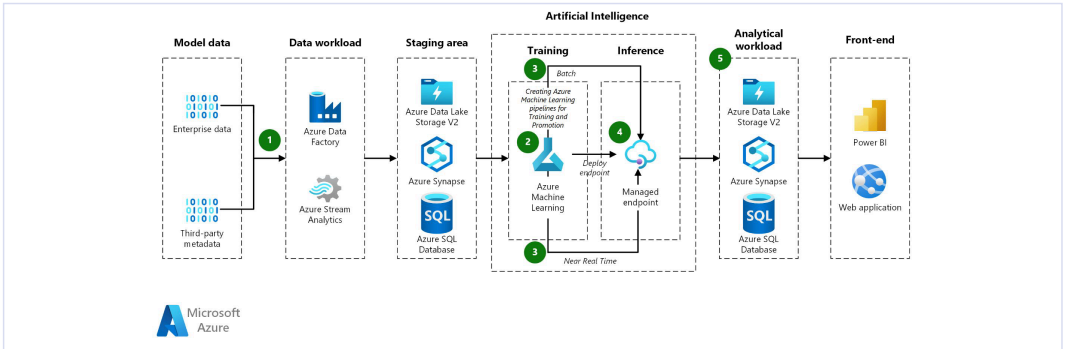
A more elaborate system linking CI/CD, data preparation, training, registry and monitoring.



Trace model training, evaluation and registration before the deployment pipeline.

Reference architecture: Azure ML

Many-models pattern: ingestion, staging, training, inference and analytical consumers.



Identify where data flows into model training and how inference feeds applications.

PART 2 / PYTHON DEPENDENCY ISOLATION

Virtual environments

Different projects. Different packages. Same computer.



What is a virtual environment?

Separate project dependencies without changing the host operating system.

ISOLATION

Dependencies

Each project can install its own Python packages and compatible versions.

WORKFLOW

Local development

Activate the environment to select Python and pip from that project.

HANDOFF

Reproducibility

Record dependencies using requirements.txt or a lockfile.

A virtual environment isolates packages; it does not virtualize the host OS or system libraries.

Inside a Python virtual environment

A .venv directory contains the isolated interpreter environment and installed packages.

DIRECTORY TREE

```
.venv/
bin/      # Scripts/ on Windows
include/
lib/python3.x/site-packages/
pyvenv.cfg
```

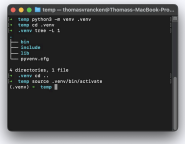
THE KEY PIECES

bin/ or Scripts/

Python and pip entry points.

site-packages/

Third-party Python packages.

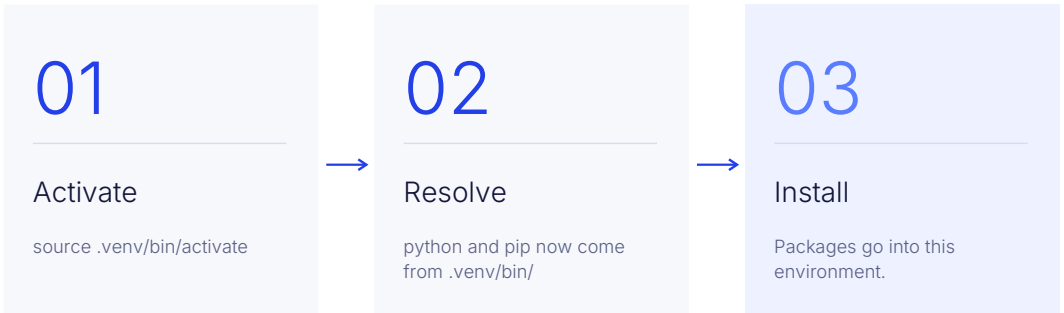


Real terminal example

POSIX example shown; Windows uses Scripts/ and Lib/site-packages. This is not a VM.

What happens when you activate .venv?

Activation changes shell PATH, so python and pip resolve inside the project.



Python virtual environments isolate Python dependencies, not arbitrary operating-system resources.

Maggie: the shared Python installation

One global installation accumulates packages from unrelated courses.



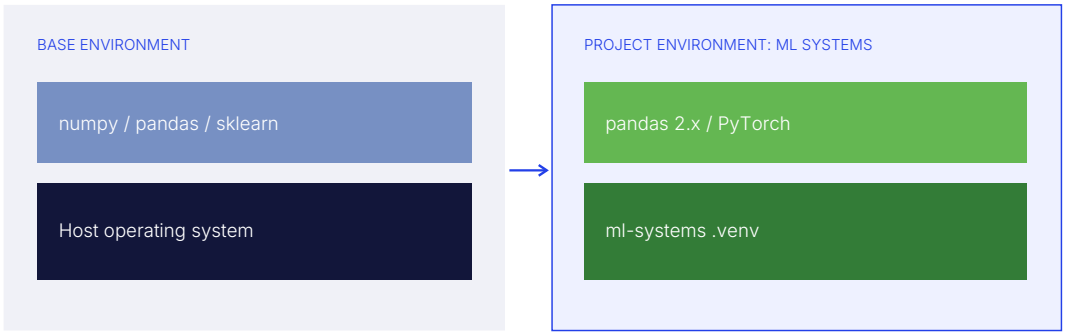
THE PROBLEM

New coursework may require a different version of a package.

Installing everything globally makes upgrades and compatibility conflicts difficult to control.

Maggie: create a project environment

A new course gets an independent set of package versions.



Package versions can differ by project while both use the same host operating system.

Maggie and John: share dependencies

Collaborators exchange a manifest, not an entire .venv directory.



Version pins aid repeatability; exact reproduction may still require OS, Python and build constraints.

John updates the dependency contract

An experiment adds a package; the manifest lets Maggie rebuild and compare.

01

John experiments

Try a new library in a separate environment.

02

Manifest changes

Record the added dependency, version and relevant code.

03

Maggie recreates

Install the update and rerun the experiment.

A requirements.txt helps, but seeds, data, OS and binary dependencies also influence reproducibility.

Using Python venv

The built-in venv module is standard for isolated project Python packages.

MACOS / LINUX TERMINAL

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install -r requirements.txt
$ python -m pip list
$ deactivate
```

KEY POINTS

venv is built into standard Python; virtualenv is a separate tool.

Windows cmd: .venv\Scripts\activate.bat; PowerShell: .\.venv\Scripts\Activate.ps1.

Install with the intended environment Python: via activation or an explicit interpreter path.

Using conda environments

Conda can manage Python and non-Python dependencies in a named environment.

CONDA WORKFLOW

```
$ conda create -n mlops python=3.12
$ conda activate mlops
$ conda install pandas
$ conda env export > environment.yml
$ conda deactivate
# On another machine only:
$ conda env create -f environment.yml
```

WHEN CONDA HELPS

Useful for projects with native compiled packages and more complex environment constraints.

Exports can contain platform pins and a local prefix; review the YAML for reuse.

Conda and venv are alternatives; choose a consistent toolchain per project.

Reproducibility starts with explicit dependencies

Do not rely on the package state of your personal laptop.

requirements.txt

```
pandas==2.2.3
numpy==2.1.1
scikit-learn==1.5.2
fastapi==0.115.0
```

CHECKLIST

- Record versions
- Track the file in Git
- Rebuild in a fresh env
- Run the same tests

Dependency manifests are a first step—not a guarantee of identical results across systems.

Virtual environments: benefits and costs

Strong default for local Python development, with a clear boundary of isolation.

BENEFITS

Why use them?

- Per-project version isolation and simpler collaboration.
- Reproducibility: record installed dependencies.
- A useful preparation step before Docker images.

LIMITATIONS

What they do not solve

- Environment setup and disk duplication.
- System libraries, OS differences and binary compatibility.
- A .venv directory is not itself a portable artifact.

Store configuration and dependency manifests in Git; normally rebuild .venv on each machine.

Use .gitignore to keep repos clean

Commit reproducibility files; ignore local environments, caches and secrets.

EXAMPLE .gitignore

```
# Python environment and caches
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/

# Local secrets
.env
secrets/
```

DO COMMIT

requirements.txt

pyproject.toml / lockfile

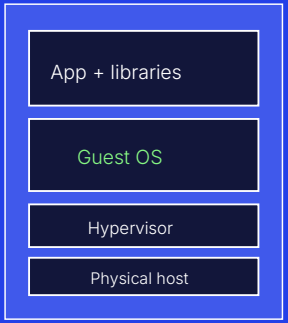
Dockerfile / application code

Ignored files may still be tracked in Git.

Ignore the generated environment, not the instructions needed to recreate it.

Virtual machines

A guest operating system on virtualized hardware.



When Python isolation is not enough

Two developers may share packages but still differ in OS libraries and binaries.

PYTHON VENV

Package-level isolation

- Shares the host OS and its system dependencies.
- Different operating systems can cause different runtime behavior.

VIRTUAL MACHINE

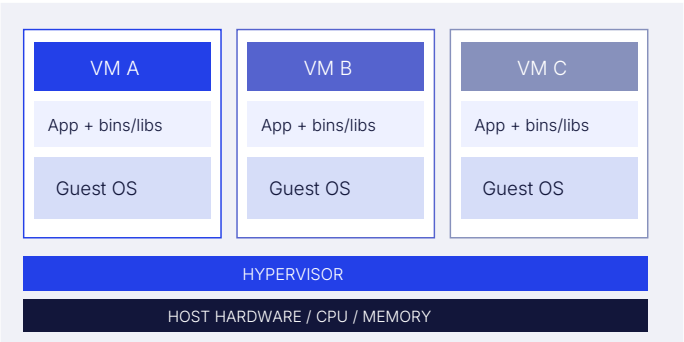
OS-level isolation

- Creates virtualized hardware and a separate guest OS.
- Lets teams reproduce a more complete operating environment.

A virtual machine is managed by a hypervisor and can run locally or in a cloud provider.

Virtual machines: the architecture

Each VM has virtual hardware resources and its own guest operating system.



VOCABULARY

Host: physical machine or base system.

Guest: OS running inside a VM.

Hypervisor: manages VMs.

Unlike a Python venv, a VM includes a guest kernel and operating system.

Why run a different operating system?

VMs can preserve older environments and test software under OS-specific constraints.

COMPATIBILITY

Legacy software

Run an older operating system required by a maintained application.

DEVELOPMENT

Cross-OS testing

Test software inside a Linux VM when the host is macOS or Windows.

SAFETY

Disposable test lab

Restore VM snapshots after an experiment or risky change.

VMs are useful far beyond running one operating system inside another as a novelty.

Virtual machines: advantages and trade-offs

Isolation is stronger than Python environments, but VMs incur guest-OS overhead.

ADVANTAGES

Flexible OS isolation

- Run guest operating systems and workloads independently.
- Snapshot, clone and automate environments.
- Consolidate multiple workloads on one physical server.

TRADE-OFFS

Resource overhead

- Guest OS consumes additional memory, disk and CPU.
- Boot and provisioning are typically slower than containers.
- Security isolation is helpful, not an absolute guarantee.

Choose VMs when OS-level separation matters; choose containers when application packaging is primary.

Where do we use virtual machines?

Local development, approved remote compute and cloud GPU/CPU infrastructure.

AWS

Amazon EC2

Select virtual CPUs, memory, GPU instances, disks and networking.

GOOGLE CLOUD

Compute Engine

Run Linux/Windows VM workloads and resize resources to demand.

MICROSOFT

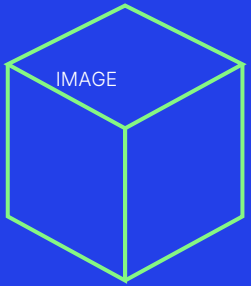
Azure Virtual Machines

Provision managed virtual hardware for workloads, dev and testing.

Approved cloud environments can process sensitive data remotely under appropriate access and residency policies.

Containerisation

Build once. Run consistently. Ship the application, not the laptop.



THE CLASSIC DEPLOYMENT PROBLEM

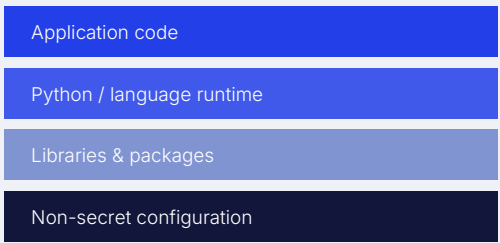
“But it works on my machine.”

Containers package a known runtime and dependencies, reducing environmental drift.

What exactly is a container?

An isolated application process with code, libraries and runtime dependencies.

APPLICATION PACKAGE



OPERATING MODEL

Containers share the kernel of their runtime host, but isolate processes, filesystems and resources.

A Linux container on macOS or Windows typically runs inside a Linux VM.

Images are portable across compatible runtimes, architectures and operating-system kernels.

Docker is a container tooling platform

Docker builds, distributes and manages container images and containers.

IMAGE CREATION

Docker Engine

Dockerfile builds; image layers; docker CLI and daemon.

RUNTIME STACK

containerd / runc

Components commonly used underneath the Docker container runtime stack.

ALTERNATIVES

Podman / LXC

Alternative tooling and Linux container technologies with different models.

Docker is popular, but containers are a broader OS-level technology and ecosystem.

Image versus container

An image specifies filesystem layers and metadata; a container is a runtime instance.

DOCKER IMAGE

Blueprint

Filesystem layers + image metadata + startup defaults.

`e.g. python-app:1.0`

RUNNING CONTAINERS

App #1

App #2

App #3

The same image can launch multiple independent processes.

Analogy: an image is a class/recipe; a container is an instance/dish. Tags may move; digests pin content.

Dockerfile → image → container

The build stage constructs an image; docker run creates a container instance.

01

Write Dockerfile

Define the base image, files, dependencies and startup.

02

Build image

docker build -t my-app:1.0 .

03

Run container

docker run --rm my-app:1.0

Rebuild the image after Dockerfile or dependency changes; containers are created from images.

Inside a Dockerfile

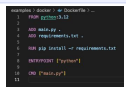
Each instruction controls a part of the application image build or startup.

ILLUSTRATIVE DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

INSTRUCTION CHEAT SHEET

FROM	Choose a base image
COPY	Add project files
RUN	Execute during build
CMD	Default run command
ENTRYPOINT	Fixed executable (optional)



Screenshot example

Use .dockerignore to exclude .env, .venv and caches from the build context.

Popular container images

Choose a trusted, maintained image that fits your runtime and security constraints.

LANGUAGES

Python / PyTorch

Official Python and framework variants for services, inference or experiments.

OPERATING BASE

Ubuntu / Alpine

Minimal user-space distributions; Alpine may introduce musl compatibility concerns.

INFRASTRUCTURE

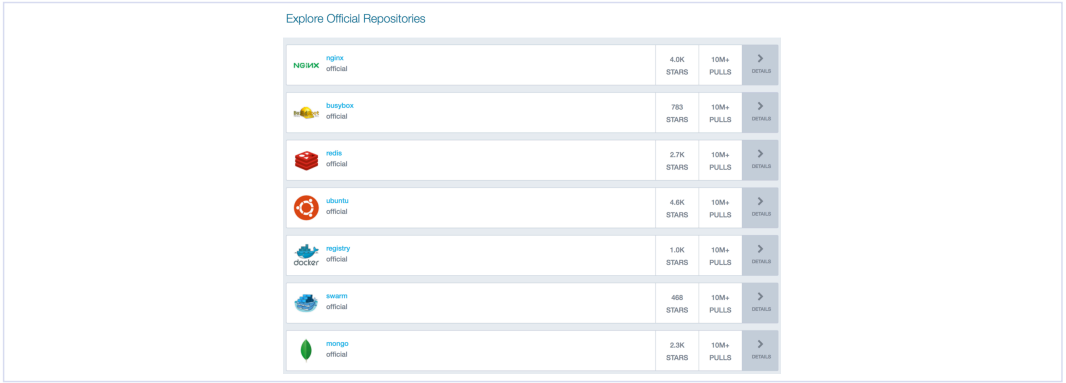
Nginx / PostgreSQL

Ready-to-configure web serving and database services; also Redis and MongoDB.

Prefer pinned tags or digests, image scanning and minimal necessary dependencies.

Official images on Docker Hub

Docker Hub lists maintained base images such as nginx, redis, ubuntu and mongo.

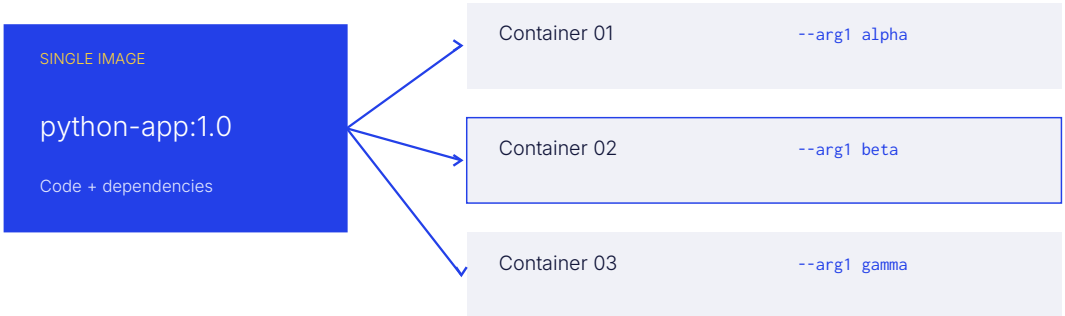


Prefer official, pinned images; check pulls and update date before you depend on one.

Screenshot: Wikimedia Commons, “ImagemDockerHub.png”, DaniloBarros, CC BY-SA 4.0.

One image can create multiple containers

Startup arguments and environment configuration can differ at runtime.



A container is an instance of an image; externalized parameters make instances behave differently.

Example: a Python CLI with arguments

The same script can produce different outputs from command-line options.

main.py — ILLUSTRATIVE

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--arg1", default="value1")
parser.add_argument("--arg2", default="value2")
args = parser.parse_args()
print(f"Argument 1: {args.arg1}")
print(f"Argument 2: {args.arg2}")
```

RUN LOCALLY

```
python main.py --arg1 hi \
--arg2 hello
```

```
Argument 1: hi
Argument 2: hello
```



Real script screenshot

Command-line inputs become process arguments; container images can expose the same interface.

Configure command defaults in Dockerfile

ENTRYPOINT defines the executable; CMD can provide overridable default arguments.

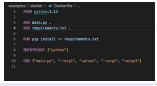
DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY main.py .
ENTRYPOINT ["python"]
CMD ["main.py", "--arg1", "value1", "--arg2", "value2"]
```

SEMANTICS

ENTRYPOINT stays as python unless explicitly overridden.

CMD supplies the default script and arguments for python.



Dockerfile screenshot

Exec-form ENTRYPOINT and CMD are JSON string arrays; runtime arguments can replace CMD.

Override CMD when starting a container

A single image produces different command-line outputs without rebuilding.

ILLUSTRATIVE TERMINAL OUTPUT

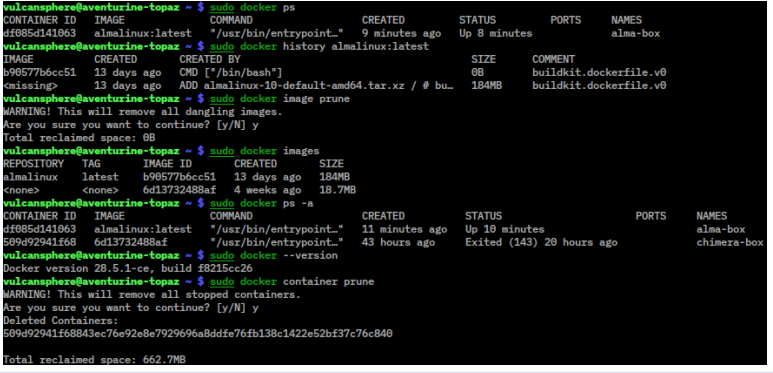
```
$ docker build -t python-imagename .
$ docker run --rm python-imagename
Argument 1: value1
Argument 2: value2

$ docker run --rm python-imagename main.py --arg1 new_value1 --arg2 new_value2
Argument 1: new_value1
Argument 2: new_value2
```

The image stays unchanged; only the container startup arguments differ.

Docker in the terminal

Listing images and containers, then cleaning up with prune.



Inspect with docker images and docker ps; clean up with the prune commands.

Screenshot: Wikimedia Commons, “Docker commands screenshot.webp”, VulcanSphere, Apache-2.0.

Docker client, host and registry

The CLI talks to a daemon; images can be pushed to or pulled from a registry.



docker build makes images; docker push/pull transfers them; docker run starts containers.

Where are Docker images stored?

Container registries host versioned image artifacts for teams and deployment systems.

PUBLIC / PRIVATE

Docker Hub

Popular hosted registry for public images and private repositories.

DEVELOPER WORKFLOWS

GHCR / Quay

Container registries that integrate with team repositories and policies.

CLOUD-MANAGED

ECR / ACR / Artifact Registry

AWS, Azure and Google-managed container image registries.

Use authenticated pushes, version tags or digests and image-provenance policies.

Registry ≠ runtime: two distinct services

Storing a container image is not the same as running a production application.

STORE THE IMAGE

Container registry

- Examples: ECR, ACR, Artifact Registry, Docker Hub.
- Holds image layers and metadata for pull/push operations.

RUN THE APPLICATION

Compute / orchestration

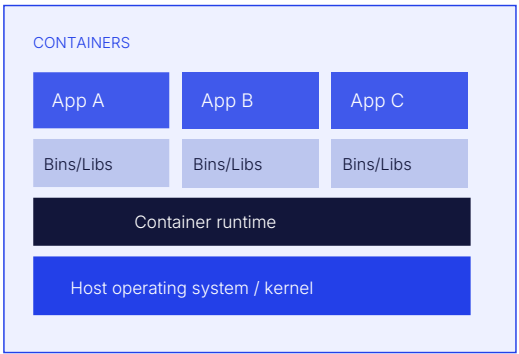
- Examples: ECS/Fargate, Cloud Run, AKS, Kubernetes.
- Allocates compute, starts containers and manages scaling.

Deployment systems pull a stored image and start one or more container instances.

Why are containers lighter than VMs?

Containers share a runtime kernel; VMs carry a separate guest OS.





Both provide isolation; VM guests virtualize an OS while containers isolate application processes.

Containers are portable—with constraints

Consistent runtime behavior depends on platform, architecture and image compatibility.

LINUX HOST

Linux container

Runs against a compatible Linux kernel with namespaces, cgroups and runtime.

MACOS / WINDOWS

Linux VM bridge

Docker Desktop / WSL2 commonly provide a small Linux VM for Linux containers.

CPU ARCHITECTURE

amd64 / arm64

Build multi-platform images or use the matching architecture.

A container image does not include its own guest kernel and cannot ignore OS/CPU boundaries.

Containers enable modular services

Package API endpoints, workers and model inference independently.

ONE APPLICATION / MULTIPLE SERVICES

API

FastAPI

```
docker image
```

Worker

batch job

```
docker image
```

Inference

model service

```
docker image
```

Orchestrators such as Kubernetes manage deployment, restart and scaling across multiple nodes.

What containers improve

A focused way to package and distribute software across development and deployment.

BENEFIT	PRACTICAL CONSEQUENCE
Portability	Reuse images on compatible runtimes and architectures.
Fast startup	Typically faster to start than an entire guest OS.
Efficiency	Share host resources instead of duplicating a guest OS.
Modularity	Deploy APIs, workers and inference independently.
Fault containment	An app crash need not terminate other containers.
Manageability	Orchestrators aid rollouts, restarts and scaling.

Use orchestration, health checks and monitoring to turn packaging into reliable operations.

Containers require security discipline

Shared kernels and untrusted images introduce operational and security risks.

ISOLATION

Shared kernel

Kernel vulnerabilities or overly privileged containers may affect a host.

SUPPLY CHAIN

Untrusted images

Images may contain vulnerable packages or malicious components.

OPERATIONS

Complexity at scale

Networking, storage, secrets and orchestration still require engineering.

Mitigate with image scanning, trusted registries, least privilege, patching and resource limits.

Container tooling matters in MLOps roles

A historical job-posting analysis highlighted the importance of Docker and Kubernetes.

HISTORICAL SURVEY

40%

of analyzed listings mentioned container tooling

READ THE CLAIM CORRECTLY

Sample: 310 MLOps job ads from Europe/US, reported Jan 2024.

Historical snapshot; not a verified 2026 labor-market rate.

Learn Docker fundamentals now; orchestration skills become useful in production MLOps.

Full overview: choose the right boundary

Virtual environments, VMs and containers address different reproducibility problems.

DIMENSION	PYTHON VENV	VIRTUAL MACHINE	CONTAINER
Isolates	Python packages	Guest operating system	App/process runtime
Kernel	Host kernel	Guest OS kernel	Runtime host kernel
Overhead	Low	Higher	Low to medium
Portability	Rebuild dependencies	VM image + compatible hypervisor	Compatible platform image
Typical use	Local Python projects	Full OS / cloud compute	Packaged deployment

Use venv for dependencies, VMs for OS isolation and containers for deployable app packages.

Lecture 3 in summary

One idea per section.

1. **Cloud service models**
   IaaS, PaaS, SaaS and FaaS trade control for convenience.
2. **Cloud ML services**
   Managed platforms and AI APIs; same building blocks everywhere.
3. **Virtual environments**
   Per-project packages; share requirements, not the .venv.
4. **Virtual machines**
   A guest OS on a hypervisor: strong isolation, more overhead.
5. **Containers**
   Isolated processes on a shared host kernel.
6. **Dockerfile, image, container**
   Build once; start many containers from one image.
7. **Registry and runtime**
   Storing an image is not the same as running it.
8. **Choosing a boundary**
   Match the isolation level to the reproducibility need.